Raft

4160 – Distributed and Parallel Computing

CUHK

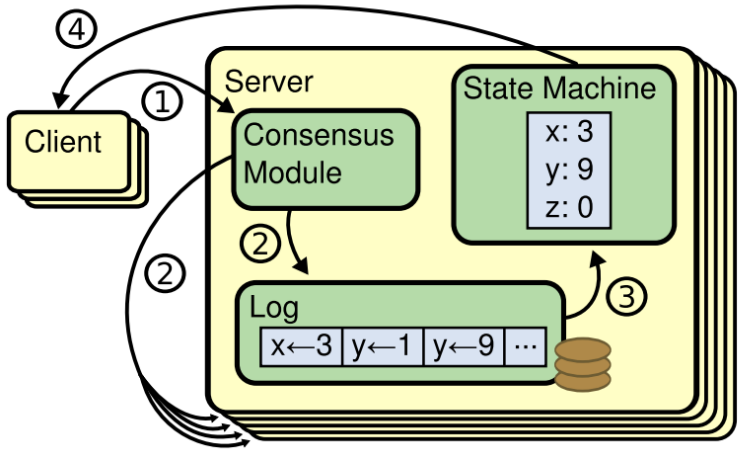
Goal: Replicated State Machine

- Replicated (high availability) Data Store
  - Serve as the high availability foundation of many systems on top
    - Single-leader, all reads have to go through the leader → performance not great → can't read any replica straight
      - If read a replica straight, may result in stale read
      - Many kv-stores often twist it to improve performance
        - » E.g. Zookeeper allows reading any replica (may result in stale read)
  - Being used in applications that HA + consistency >> performance
    - E.g., Directory service in the cloud (maintaining the VM-to-IP list)
      - Not throughput demanding, pretty static
- Setting
  - Fail-stop (not Byzantine), lost/delayed message

Goal: Replicated State Machine

- RSM vs Broadcasting
  - RSM: agree on the **final state**
  - Broadcasting: agree on the **ordering of the input**
    - Under multi-core: same input != same output
      - If play the input *serially* → agreed on the final state
      - If play the input in parallel → different final states (thread race from OS scheduling)
- RSM is a (popular) instance of Consensus
- Raft is (multi-)Paxos variant
  - Multi-Paxos:
    - no need to elect new leaders for every new command to apply to the RSM
    - Just re-use the leader if the leader is not dead (skip Phase 1)
    - https://www.alibabacloud.com/blog/paxos-raft-epaxos-how-has-distributed-consensus-technology-evolved_597127

Key idea



– Every replica

1. Gets the same input sequence // Replicated Input Log
2. Applies an input command **using single-thread**
3. Gets the same output state

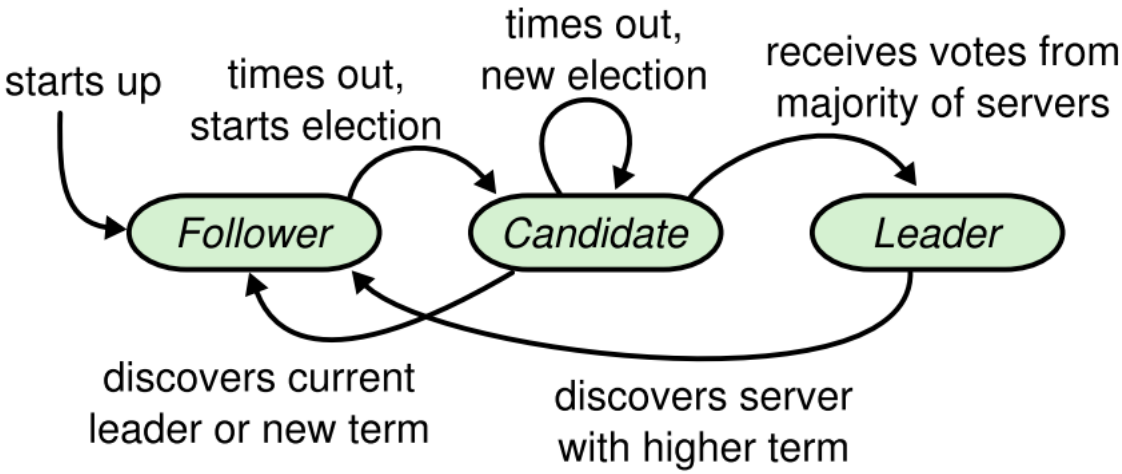
- Step 1 is the key ➔ consensus
  - Buffer the log entry (input command) until all replicas agree on that
    - » Erred log entry could be discarded/corrected by subsequent operations that piggyback the synchronization info
  - Hence, the key step is essentially 'atomic broadcast'

Raft

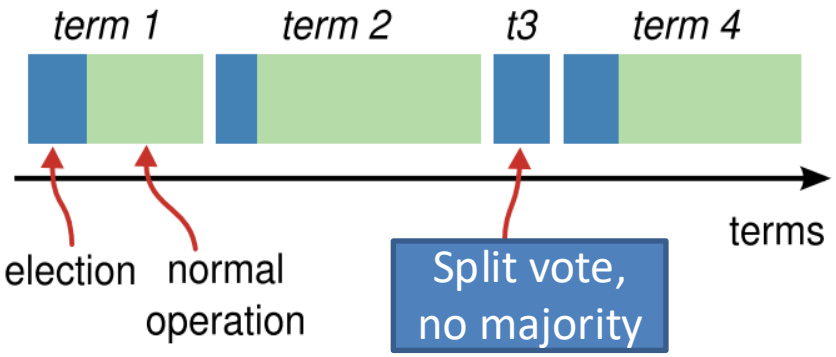
- Break the problem into:
  - Leader-election
  - Log-replication

Server states (roles)

- At any given time, each server is either:
  - Leader:
    - Handle all client interactions
    - Operate until it fails
  - Follower:
    - completely passive as a replica only
    - If client X directly contacts it, tells X who the leader L is
  - Candidate:
    - discover leader dies, starts election
- Normal case: 1 leader, n-1 followers

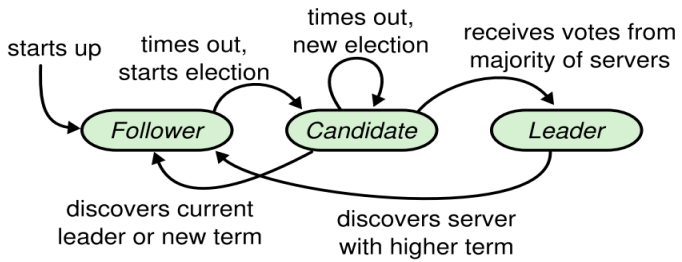


Time divided into “terms”



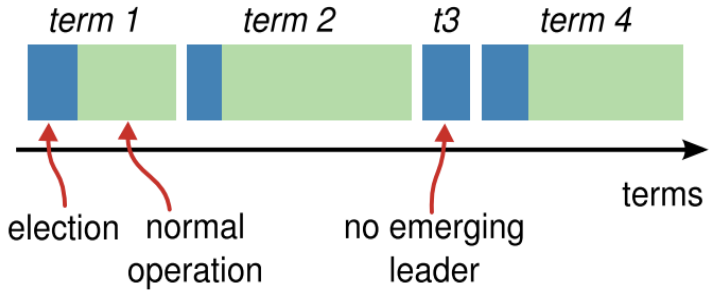
- Each term has 2 parts:
  - Begins with an election
  - Normal operation (with the elected leader)
- Some terms have no leader (e.g., split vote)
- Each replica maintains its own **currentTerm** value
  - Useful for knowing itself is out-of-date or not
  - E.g., server 1 is in term 1945, but it receives an RPC from server 9 saying that its term 2026

Follower f



- Everybody starts up as a follower
- Follower f expects to receive RPCs from leader or candidates
  - Leader would:
    - f.**appendEntries**(command x, …)
    - f.**appendEntries**(null, …) //leader's heartbeat in case no cmd recently
    - Return <success?, follower's currentTerm>
  - Candidate would: //could be more than 1 candidate
    - f.**requestVote**(…)
    - Return <yes/no, follower's currentTerm>
  - If no RPC from leader after **electionTimeout**
    - Assume leader is dead
    - f becomes a candidate and triggers a new election

When f triggers an election



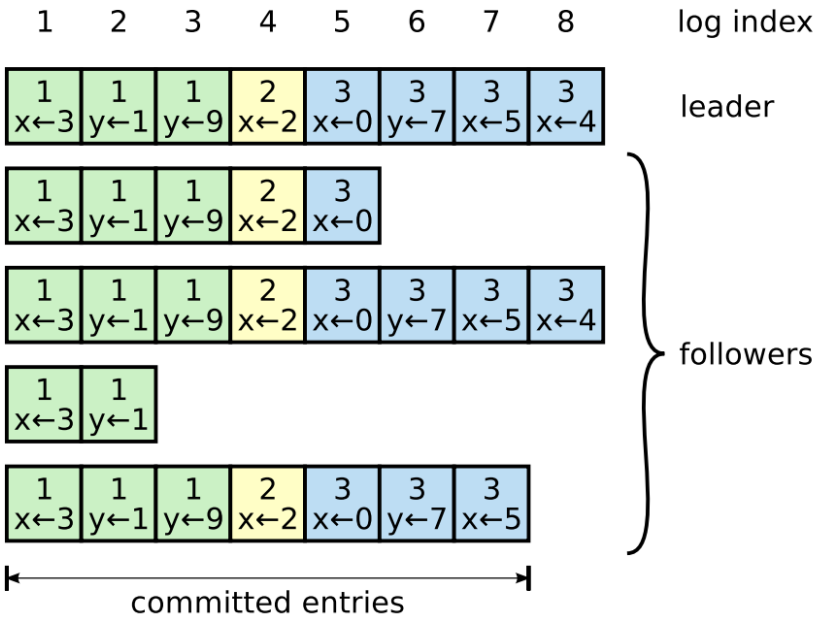
1. my.currentTerm++
2. my.state = candidate
3. voteformyself
4. For all other servers s:
   s.requestVote(my id);
5. **If** I get the majority votes
   a) my.state = leader
   b) For all other servers: s.appendEntries(my id, null) //send heartbeat

Else If I get appendEntries from somebody L else
   my.state = follower
   my.myleader = L

Else split vote
   restart from #1.

= announcement of new leader

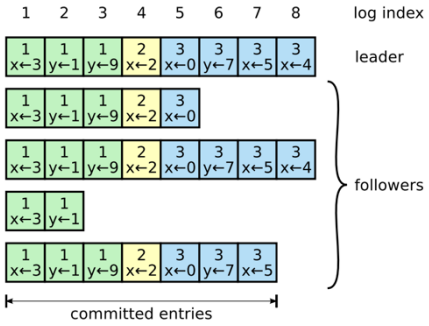
Log Replication



- Log Entry=<term, update cmd>*
- Log stored on persistent storage
- A log entry is "committed" if it is known be to on ***a majority of servers***
  - **commitIdx++**
  - once committed, eventually executed by every replica

*well, you won't log the read commands that don't change the state

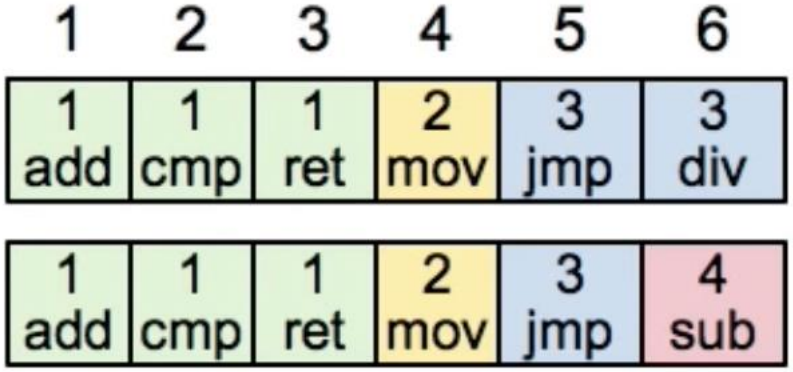
Leader L



- Clients send command to the leader
- Leader appends the command to its own log
- Leader broadcasts the command to its follower f
  - f.appendEntries(command)
- Once a command is committed
  - L.apply(command)
  - Reply client: done!
  - In subsequent f.appendEntries(cmd, commitIdx)
    - followers refer commitIdx to apply the committed commands
- Leader re-posts committed entries to followers if they are slow, lost message, join, reborn, etc.

Raft is designed to uphold the following at all times

- Same term in the same log entry #?
  - ➔Must be the same command
  - ➔All preceding entries must be the same as well



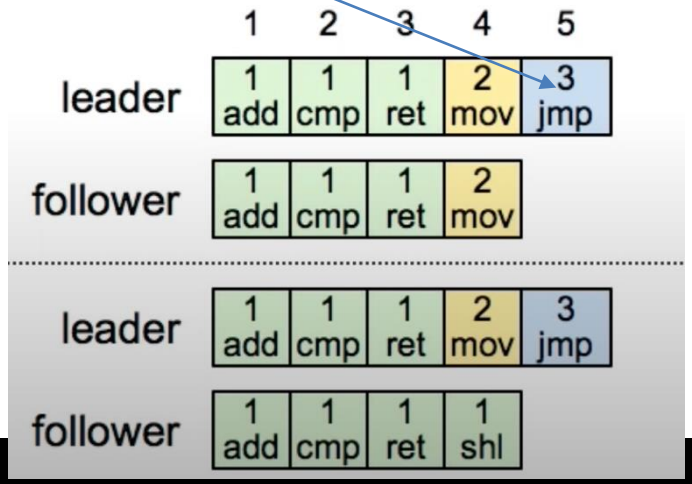
- Once an entry is committed, all preceding entries are also committed

But how to uphold that in the protocol?

- via AppendEntries piggybacked info
- When L sends appendEntries(

  command x, **term&index** of my preceding entry)

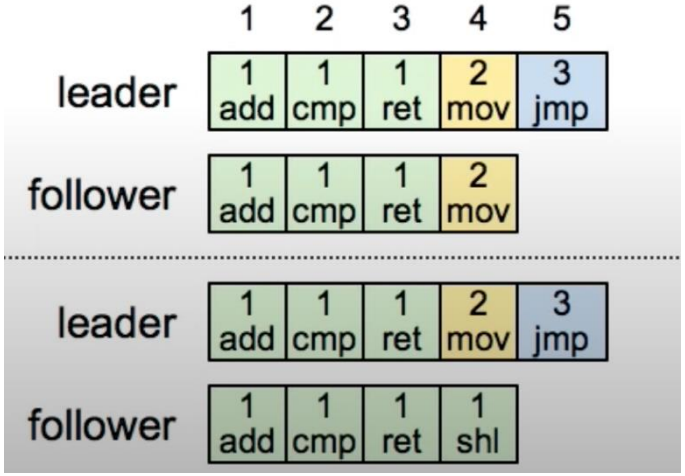
  – E.g., `appendEntries(command 3jmp,` **`term=2&index=4)`**
- Follower f must check if its own **term&index** match L's
  – Reply fail if they don't match



But how to uphold that in the protocol?

- If appendEntries fail, leader will retry by sending some **earlier entries** to **overwrite the inconsistent** uncommitted ones until the two (leader & follower) can match up again
- In the example, L's f.**nextIndex--** and attempts to

```
appendEntries(command 2mov, 1&3)
```



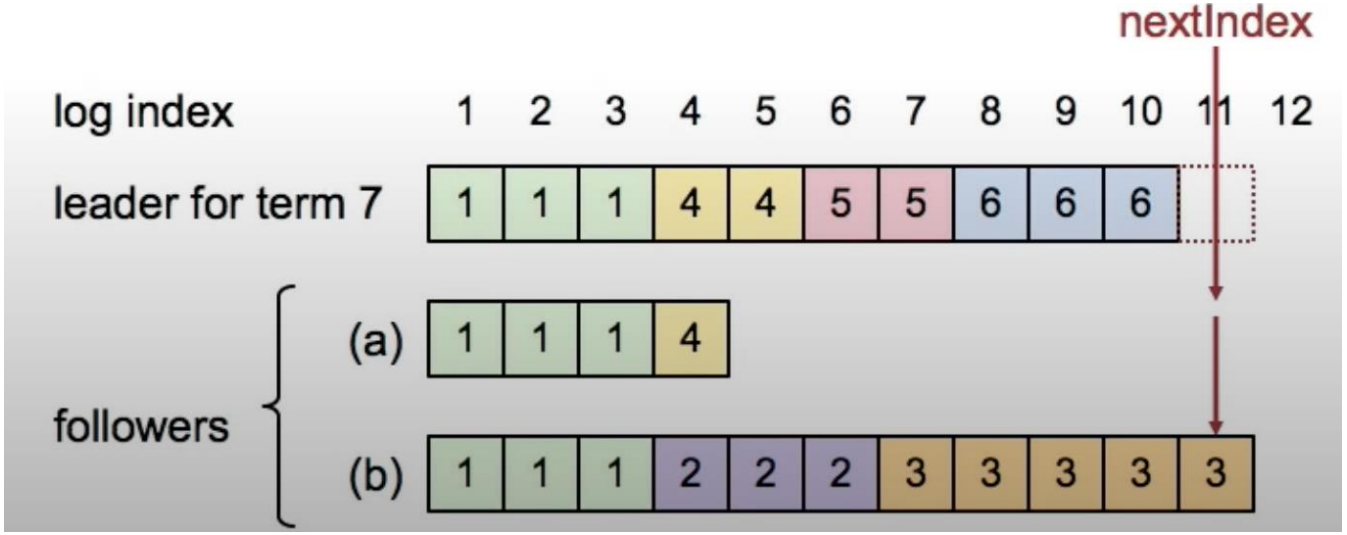
Repairing the log inconsistency – implementation

- Leader keeps *nextIndex* **for each follower**

  ➔ the next entry I should send to f

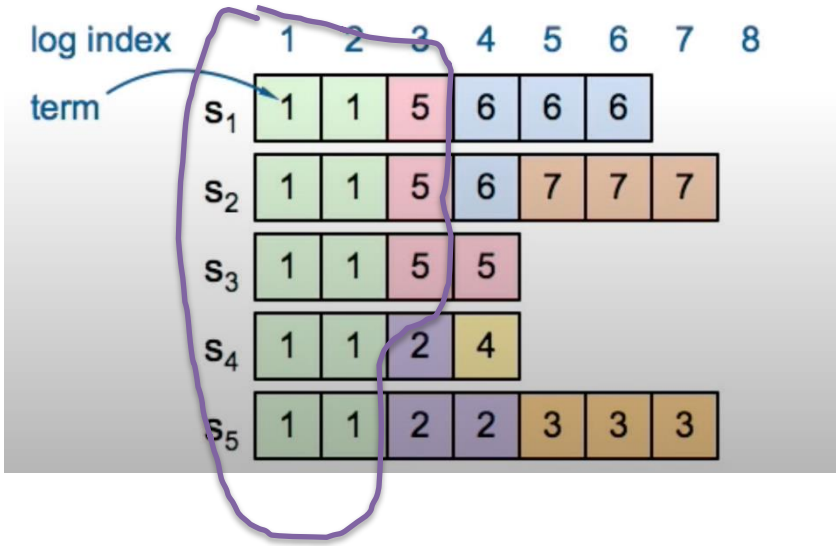
  – If I appendEntry to f and get rejected

    - f.nextIndex--



Reconciling the replicated logs

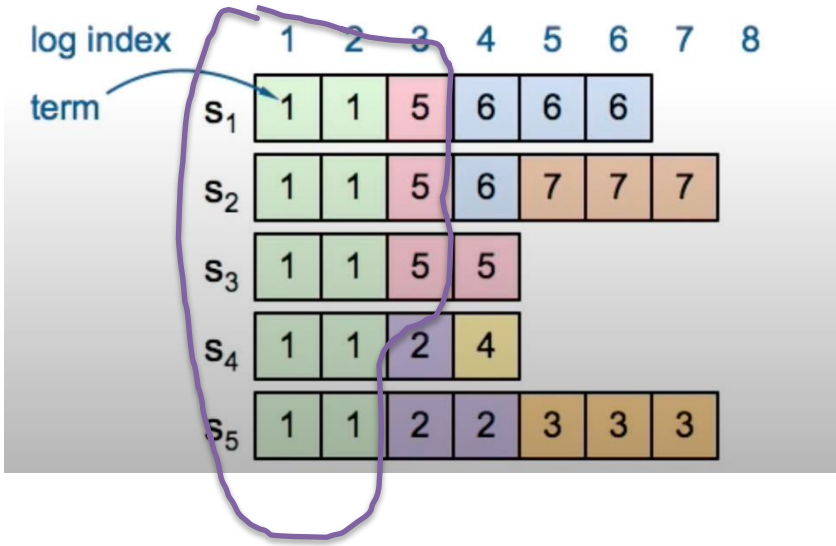
- Example inconsistency:
  - Old leader dies with some but not all entries replicated (i.e., uncommitted yet)
- New leader is always the single source of truth
- It will make all followers to converge to its log progressively
- In the example, all entries except those committed ones are dispensable



- Say, S2 becomes the new leader, eventually everyone will get the same log as S2

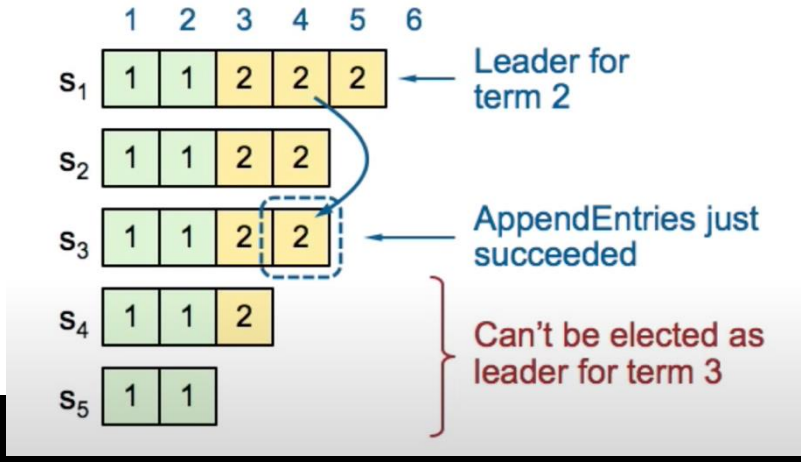
Eligibility of becoming a candidate

- If S2 dies, can S4/S5 be a leader?
  - No! They don't have all the committed entries!
- So, if a candidate C with a less complete log contacts me for a vote, I will say no to C (because I am better than C) and I promote myself be the candidate instead
  - My latest term > yours? Reject
  - Our terms are the same but my log is longer than yours? Reject



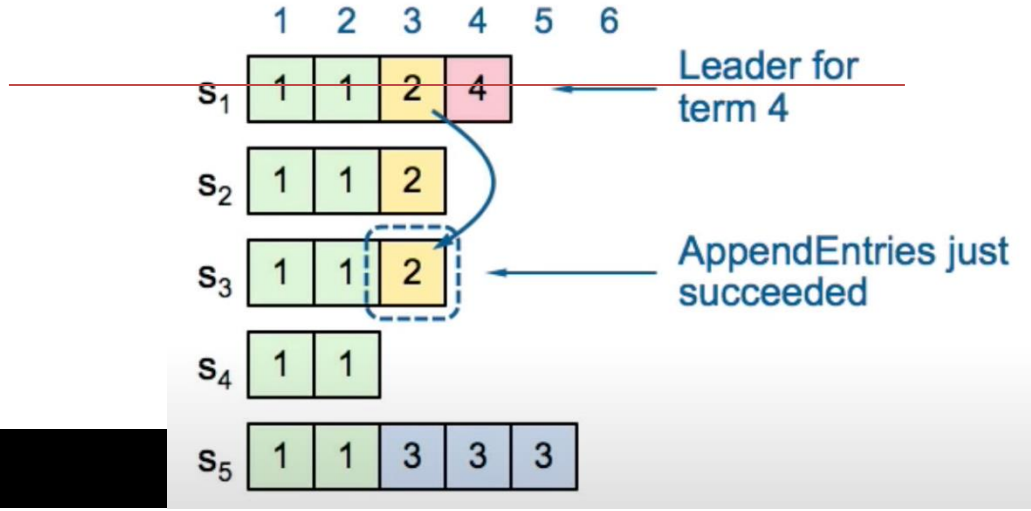
Leader election

- Who can be the leader of term 3?
- S5: can't
  - S1-S4 are longer and have new term than it;
    - they won't vote for S5
- Idx 4 has also committed
- Only the ones with idx=4 and term=2 can win the election



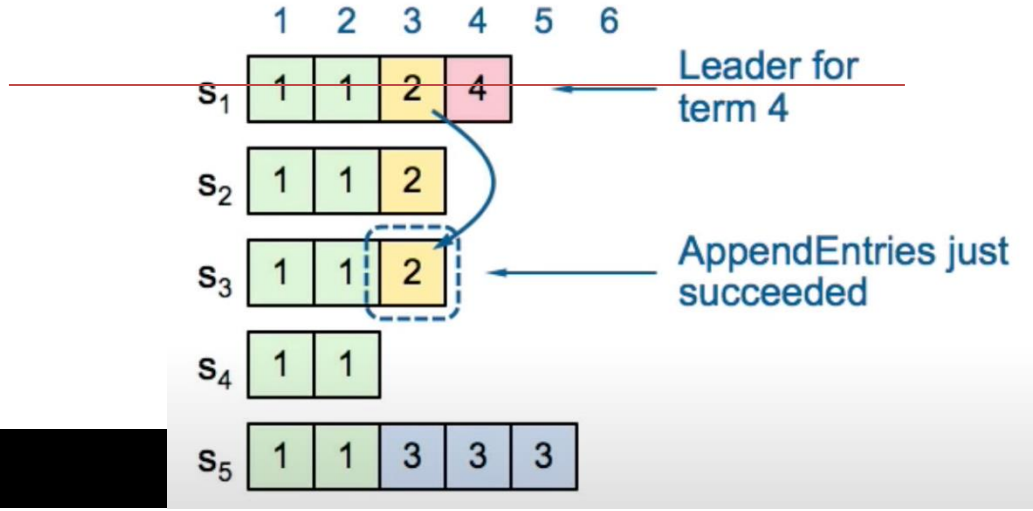
Leader election bug?

- If S1 dies, could S5 become a new leader for term 5?



Leader election bug?

- If S1 dies, could S5 become a new leader for term 5?
  - Yes. Newer term, longer log!
  - And S5 will get the votes and overwrite "committed" 2 on S1 to S3!
- So, the definition of 'commit' shall be fine-tuned



The commit rule

- For a log entry in the current term (leader term),
  - the leader commits it once the entry is stored on a majority.
- For log entries in previous terms, the leader uses the following commit rule

	A	B	C	D	E
S1	1	1	2	2	
S2	1	1	2	2	3
S3	1	1			
S4	1	1			
S5	1	1			

New Leader (Term 3), trying to appendEntries C&D from previous term 2

The commit rule

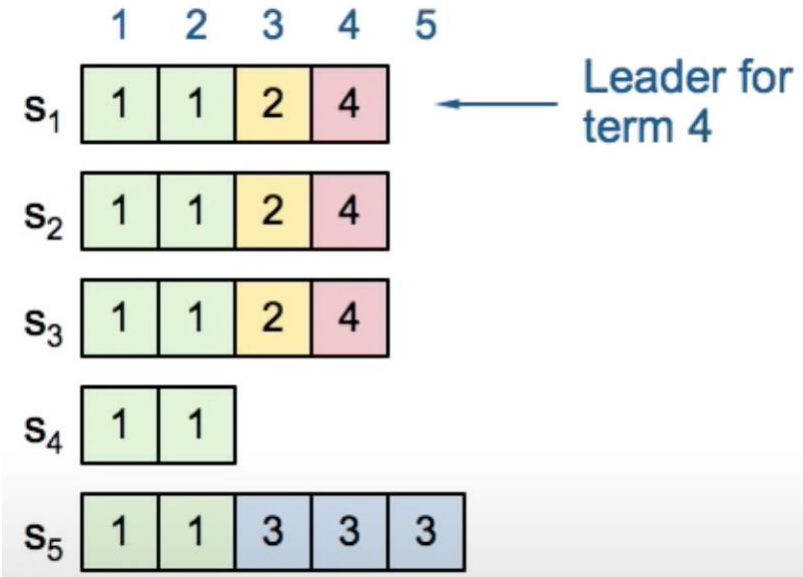
- For a log entry in the current term (leader term),
  - the leader commits it once the entry is stored on a majority.
- For a log entry in previous terms,
  - the leader commits it when majority + at least one entry in leader's term is also committed

	A	B	C	D	E
S1	1	1	2	2	
S2	1	1	2	2	3
S3	1	1	2	2	
S4	1	1			
S5	1	1			

C&D are still not regarded as committed from leader S2's point-of-view until E is also committed

Leader election bug?

- 2 is not committed until 4 is also committed
- Now, once 2 is committed
  - even S1 is dead
    - S2 and S3 won't vote S5 as the new leader
    - Yellow entry is safe and won't shock the client



The split-brain problem

- An old leader might not be dead, but just be temporarily network partitioned and come back
- Now we have 2 “leaders”, where the old one also thinks it is the leader and send appendEntries…
- Term update rule (for everyone):
  - if (I send any RPC but is rejected because your term is newer than mine or I receive any RPC whose sender’s term is newer than mine)
    - I update my term and become a follower

Client Protocol

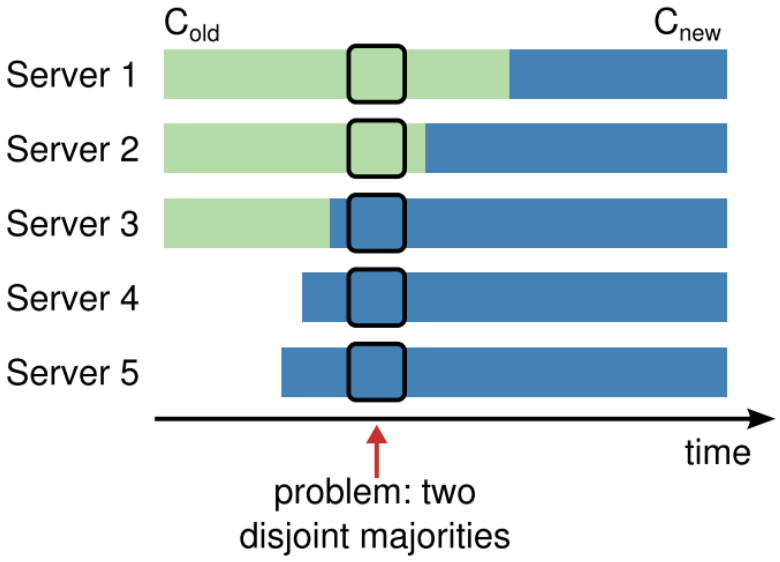
- What if

Leader	network	Client
Commit your cmd x		
Reply client ok	Lost reply msg	?
Apply x		timeout!
		Resend cmd x

- Client makes a uniqueID (say, IP address + Seq ID) to uniquely identify each cmd
- Server can then ignore duplicated command
- If client doesn't crash → exactly-one semantic

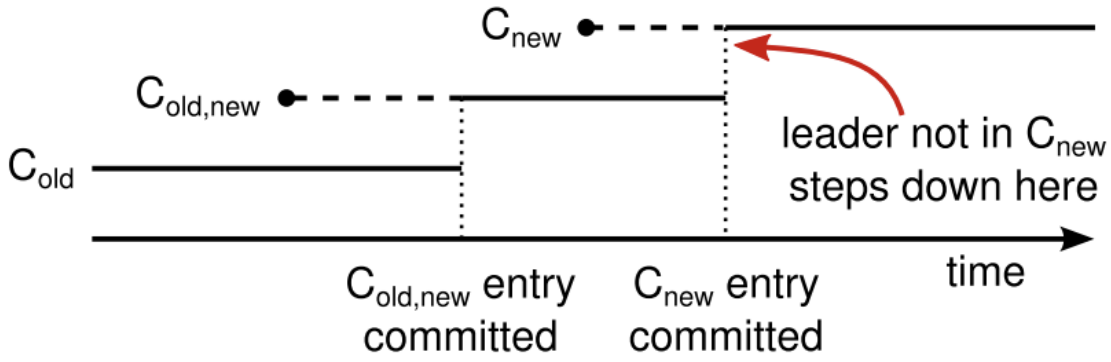
Configuration changes

- E.g., change the replication factor from 3 to 5
  - Everybody switches to the new config at different speed
- Conflicting majority problem:
  - Servers 1&2 are not ready, their majority def'n is 2 out of 3
  - Servers 3-5 are ready, their majority def'n is 3 out of 5
  - During transition, may result in two leaders
    - (e.g., S2 elected with votes from S1&S2; S4 elected from votes S3,S4,S5)



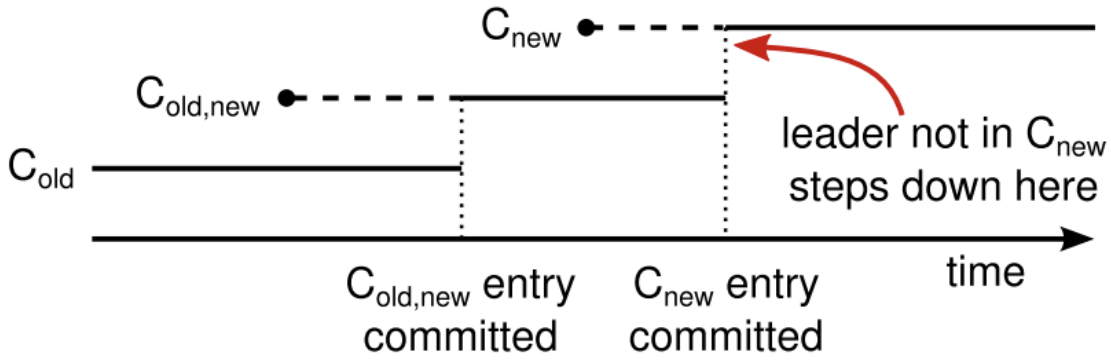
Solution: 2-phase

- Add **special** config change "commands"
  - How special?
    - Replicas apply (accept) the new config immediately, w/o waiting for committed
- Phase 1:
  - On view change, leader applies and appendEntries($C_{old+new}$) to all nodes in C_{old} U C_{new}
- Phase 2:
  - Once the $C_{old+new}$ **is committed**, leader broadcasts command C_{new}
  - When C_{new} **is committed**, transition finishes



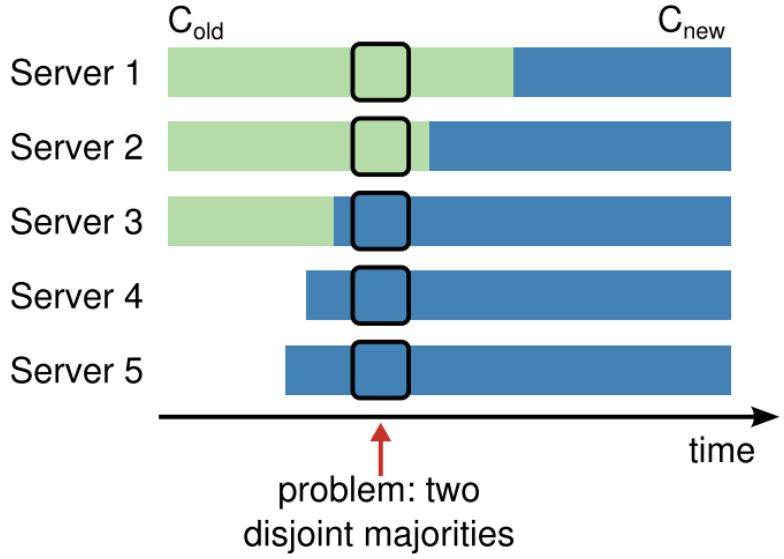
Joint-consensus

- Nodes that have applied $C_{old+new}$ are in *joint-consensus mode*
- Under this mode
  - Any node in C_{old} or C_{new} can be the leader
  - Agreement (for elections and entry commitment) requires majorities from **both** C_{old} and C_{new}



Decision maker(s)

- Before C_old+new committed
  - Decision maker from C_old
- After C_old+new committed
  - joint-consensus mode
    - any leader elected under C_old+new must overlap with both old and new groups
- After C_new committed
  - C_new (everyone needs 3 out of 5)



Difficulties of implementing Raft

- Assume 5 nodes: L, A, B, C, D
- Case 1:
  - Leader has added “appendEntries(key1, value-apple)” to its log
    - appendEntries to the followers
  - Nodes A and B
    - Replied <append log ok>
  - Nodes C and D
    - No reply……
- Case 2:
  - …